SmartSpend

Personal Finance & Expense Tracker

Bug Hunting Competition — Official Master Bug Manifest & Solutions

Generated: October 21, 2026 | Version 1.0.0

CONFIDENTIAL COMPETITION MATERIAL — FOR ORGANIZERS ONLY

This document contains the complete intentional defects catalog, reproduction steps, root causes, and surgical code fixes for the SmartSpend bug-hunting challenge. Keep strictly private until the evaluation phase concludes.

Defect Distribution Summary

Total Documented Intentional Defects: 15 | Master Toggle: BUG_CHALLENGE_MODE=true / false

ID	Title	Level	Component
BUG-BEG-01	Transaction Search Case Sensitivity	**Beginner**	Transactions
BUG-BEG-02	Transaction Amount Sort Order Invers	**Beginner**	Transactions
BUG-BEG-03	Zero Amount Expense Transaction Acce	**Beginner**	Transaction
BUG-BEG-04	Filter Empty State Shows Initial Emp	**Beginner**	Transaction
BUG-BEG-05	Profile Display Name Untrimmed White	**Beginner**	Settings
BUG-INT-01	Monthly Savings Sums Expenses Instea	**Intermediate**	Dashboard
BUG-INT-02	Budget Progress Bar Defective Over-B	**Intermediate**	Budget
BUG-INT-03	Date Range End-Date Boundary Exclude	**Intermediate**	Transactions
BUG-INT-04	Category Spending Chart Excludes "Bi	**Intermediate**	Dashboard
BUG-INT-05	Transaction Edit Leaves Dashboard Ne	**Intermediate**	Global
BUG-ADV-01	30-Day Fixed Offset Excludes 31st Da	**Advanced**	Monthly
BUG-ADV-02	Budget Utilization Subtracts Categor	**Advanced**	Budget
BUG-ADV-03	Category Line-Item Floor Rounding Di	**Advanced**	Monthly
BUG-ADV-04	Filtered Pagination Calculates Total	**Advanced**	Transactions
BUG-ADV-05	Tablet Viewport (768px) Transaction	**Advanced**	Responsive

BUG-BEG-01: Transaction Search Case Sensitivity

Difficulty: **Beginner** | Severity: Low | Affected: Transactions Page (TransactionController)

Reproduction Steps (from UI):

1. Open the Transactions page.
2. Locate a transaction with title "Grocery Store Run".
3. In the search input, type "grocery" in lowercase.
4. Observe that the search returns 0 matching results.

Expected Behavior:

Search should be case-insensitive, matching "Grocery" when searching for "grocery".

Actual Defective Behavior (Competition Mode):

Case-sensitive regex query fails to match mixed or uppercase text.

UI Recognition Clues:

Searching for lowercase common words returns empty results even when matching items exist in table.

Root Cause Analysis:

Regex query in TransactionController lacks the case-insensitive "i" option ($options: "i").

Exact Remediation / Solution:

Use regex with case-insensitive option: filter.title = { $regex: escapeRegex(search.trim()), $options: "i" };

Corrected Behavior:

Transactions matching the query regardless of uppercase or lowercase are returned.

Automated Regression Test:

server/src/tests/competition_bugs.test.js -> BUG-BEG-01

Post-Fix Verification:

Type "grocery" and verify "Grocery Store Run" appears in search results.

BUG-BEG-02: Transaction Amount Sort Order Inversion

Difficulty: **Beginner** | Severity: Low | Affected: Transactions Page (TransactionController)

Reproduction Steps (from UI):

1. Go to Transactions page.
2. Select "Sort by Amount" from the sort dropdown.
3. Verify direction button indicates "!‘ Ascending".
4. Observe that the highest amount transaction appears at the top.

Expected Behavior:

Ascending sort should place smallest amounts first.

Actual Defective Behavior (Competition Mode):

Ascending sort places largest amounts first, inverting expected behavior.

UI Recognition Clues:

Clicking "Ascending" orders from highest to lowest amount.

Root Cause Analysis:

Inverted ternary assignment in TransactionController: sortOptions.amount = order === 1 ? -1 : 1.

Exact Remediation / Solution:

Set sortOptions.amount = order;

Corrected Behavior:

Ascending sorts low-to-high; descending sorts high-to-low.

Automated Regression Test:

server/src/tests/competition_bugs.test.js -> BUG-BEG-02

Post-Fix Verification:

Sort by amount ascending and check that smaller amounts appear on page 1.

BUG-BEG-03: Zero Amount Expense Transaction Accepted

Difficulty: **Beginner** | Severity: Medium | Affected: Transaction Modal (TransactionForm & TransactionController)

Reproduction Steps (from UI):

1. Click "+ Add Transaction".
2. Fill in title "Sample Free Item", Category "Groceries", Amount "0.00".
3. Click "Add Transaction".
4. Transaction is saved and appears in table with amount ¹0.00.

Expected Behavior:

Financial records must require amount > 0.

Actual Defective Behavior (Competition Mode):

Form and API accept ¹0 transactions without rejection.

UI Recognition Clues:

Users can submit ¹0.00 transactions which clutter financial records.

Root Cause Analysis:

Validation condition checks numAmount < 0 instead of numAmount <= 0.

Exact Remediation / Solution:

Update validation to reject if isNaN(numAmount) || numAmount <= 0.

Corrected Behavior:

Amount = 0 triggers validation error "Amount must be greater than 0".

Automated Regression Test:

server/src/tests/competition_bugs.test.js -> BUG-BEG-03

Post-Fix Verification:

Attempt to submit ¹0 amount and verify error message prevents submission.

BUG-BEG-04: Filter Empty State Shows Initial Empty Message

Difficulty: **Beginner** | Severity: Low | Affected: Transaction Table (TransactionTable.jsx)

Reproduction Steps (from UI):

1. Navigate to Transactions page with existing records.
2. Enter search query "nonexistent123".
3. Notice empty state says "No transactions recorded yet" instead of "No matching transactions found".

Expected Behavior:

Should distinguish between an empty database and an empty filtered search result.

Actual Defective Behavior (Competition Mode):

Shows generic onboarding empty state even when active filters caused the empty list.

UI Recognition Clues:

Searching for non-existent items shows "Start tracking your financial journey" onboarding text.

Root Cause Analysis:

TransactionTable empty state condition ignores hasActiveFilters in competition mode.

Exact Remediation / Solution:

Check hasActiveFilters to render "No matching transactions found" and offer "Reset Filters" action.

Corrected Behavior:

Shows search-specific empty state with clear button.

Automated Regression Test:

client/src/components/TransactionTable.jsx

Post-Fix Verification:

Type non-existent search and observe "No matching transactions found" message.

BUG-BEG-05: Profile Display Name Untrimmed Whitespace

Difficulty: **Beginner** | Severity: Low | Affected: Settings & Profile (ProfileController.updateProfile)

Reproduction Steps (from UI):

1. Navigate to Settings & Profile page.
2. In Full Name, input "Alex Sharma " (3 trailing spaces).
3. Click "Save Profile Changes".
4. Return to Dashboard: Greeting displays "Welcome back, Alex Sharma !".

Expected Behavior:

Names should be trimmed before saving to ensure clean presentation.

Actual Defective Behavior (Competition Mode):

Saves raw untrimmed string with trailing whitespace.

UI Recognition Clues:

Trailing spaces visible before exclamation marks in dashboard greetings.

Root Cause Analysis:

Missing .trim() on name in profileController.js.

Exact Remediation / Solution:

Use user.name = name.trim(); before saving.

Corrected Behavior:

Trailing and leading spaces are automatically trimmed.

Automated Regression Test:

server/src/tests/competition_bugs.test.js -> BUG-BEG-05

Post-Fix Verification:

Update profile with trailing spaces and verify greeting displays cleanly.

BUG-INT-01: Monthly Savings Sums Expenses Instead of Subtracting

Difficulty: **Intermediate** | Severity: High | Affected: Dashboard Overview (DashboardController.getSummary)

Reproduction Steps (from UI):

1. Open Dashboard with Income = ¹85,000 and Expenses = ¹35,000.
2. Inspect "Current Month Savings" card.
3. The card displays ¹1,20,000 instead of ¹50,000.

Expected Behavior:

Monthly Savings = Current Month Income - Current Month Expenses.

Actual Defective Behavior (Competition Mode):

Dashboard calculates Income + Expenses, inflating savings display.

UI Recognition Clues:

Savings card value exceeds total income whenever expenses are logged.

Root Cause Analysis:

Addition operator (+) used instead of subtraction (-) in dashboardController.js.

Exact Remediation / Solution:

Calculate currentMonthSavings = Math.round((currentMonthIncome - currentMonthExpenses) * 100) / 100;

Corrected Behavior:

Savings correctly displays net difference between income and expenses.

Automated Regression Test:

server/src/tests/competition_bugs.test.js -> BUG-INT-01

Post-Fix Verification:

Verify Current Month Savings equals Income minus Expenses.

BUG-INT-02: Budget Progress Bar Defective Over-Budget Width Clamp

Difficulty: **Intermediate** | Severity: Medium | Affected: Budget Cards (BudgetCard.jsx)

Reproduction Steps (from UI):

1. Create budget for "Dining Out" with limit ¹5,000.
2. Record expenses totaling ¹6,000 (120% utilized).
3. Observe progress bar: width shrinks to 20% instead of filling 100% with red alert styling.

Expected Behavior:

Exceeded budget progress bar should fill 100% width with danger red color.

Actual Defective Behavior (Competition Mode):

Bar shrinks to surplus percentage ((spent - limit) / limit * 100).

UI Recognition Clues:

Over-budget cards have progress bars that look almost empty.

Root Cause Analysis:

Defective overflow calculation in BudgetCard.jsx.

Exact Remediation / Solution:

When isOverBudget is true, clamp progressWidth = 100;

Corrected Behavior:

Progress bar fills completely with red danger indicator.

Automated Regression Test:

client/src/components/BudgetCard.jsx

Post-Fix Verification:

Exceed a budget limit and verify progress bar stays at 100% full width in red.

BUG-INT-03: Date Range End-Date Boundary Excludes Same-Day Records

Difficulty: Intermediate | Severity: Medium | Affected: Transactions Filter (TransactionController.getTransactions)

Reproduction Steps (from UI):

1. Create transaction on 2026-10-15 at 14:00.
2. Filter transactions: Start Date = 2026-10-10, End Date = 2026-10-15.
3. Transaction on 2026-10-15 is omitted from results.

Expected Behavior:

End date filter should include records through 23:59:59.999 of the specified day.

Actual Defective Behavior (Competition Mode):

End date boundary truncated to midnight 00:00:00, omitting afternoon transactions.

UI Recognition Clues:

Filtering up to "today" hides transactions recorded earlier today.

Root Cause Analysis:

d.setHours(0, 0, 0, 0) used instead of d.setHours(23, 59, 59, 999).

Exact Remediation / Solution:

Set end boundary: d.setHours(23, 59, 59, 999); filter.transactionDate.$lte = d;

Corrected Behavior:

Transactions occurring anytime on the end date are included.

Automated Regression Test:

server/src/tests/competition_bugs.test.js -> BUG-INT-03

Post-Fix Verification:

Filter with end date matching today and verify same-day transactions appear.

BUG-INT-04: Category Spending Chart Excludes "Bills" Category

Difficulty: **Intermediate** | Severity: Medium | Affected: Dashboard Charts (DashboardController.getCharts)

Reproduction Steps (from UI):

1. Record expenses in "Bills" category.
2. Inspect "Category Spending" donut breakdown on Dashboard.
3. "Bills" is absent, and the sum of displayed categories is less than total expenses.

Expected Behavior:

All expense categories should appear in the category breakdown chart.

Actual Defective Behavior (Competition Mode):

MongoDB aggregation stage explicitly excludes Bills category.

UI Recognition Clues:

Category chart total does not equal Monthly Expenses KPI card.

Root Cause Analysis:

categoryPipeline includes match { category: { $ne: "Bills" } } in competition mode.

Exact Remediation / Solution:

Remove the $ne: "Bills" filter stage from category aggregation.

Corrected Behavior:

Bills category appears with correct amount and percentage in chart.

Automated Regression Test:

server/src/tests/competition_bugs.test.js -> BUG-INT-04

Post-Fix Verification:

Verify Bills appears in the Category Spending chart.

BUG-INT-05: Transaction Edit Leaves Dashboard Net Balance Stale

Difficulty: **Intermediate** | Severity: Medium | Affected: Global State / Dashboard Sync (AuthContext.jsx)

Reproduction Steps (from UI):

1. Note Dashboard Net Balance (e.g. ¹50,000).
2. Navigate to Transactions page and edit an expense from ¹1,000 to ¹10,000.
3. Return to Dashboard: Net Balance still shows ¹50,000 until full browser reload.

Expected Behavior:

Editing a transaction should immediately trigger dashboard data refresh.

Actual Defective Behavior (Competition Mode):

triggerDashboardRefresh(true) suppresses refresh event on edit actions.

UI Recognition Clues:

Dashboard numbers only update on page reload after editing transactions.

Root Cause Analysis:

Condition in AuthContext skips setDashboardKey increment on edits.

Exact Remediation / Solution:

Always execute setDashboardKey(prev => prev + 1); regardless of action source.

Corrected Behavior:

Dashboard balance reflects edited transaction immediately upon return.

Automated Regression Test:

client/src/context/AuthContext.jsx

Post-Fix Verification:

Edit transaction amount, return to dashboard, and check updated balance.

BUG-ADV-01: 30-Day Fixed Offset Excludes 31st Day from Monthly Reports

Category: **Advanced** | Severity: High | Affected: Monthly Reports (ReportController.getMonthlyReport)

Reproduction Steps (from UI):

1. Record an expense on October 31st (e.g. ¹1,500 Shopping).
2. Open Reports page for October 2026.
3. Observe Total Expenses: October 31st transaction is completely missing.

Expected Behavior:

Monthly report must encompass all 31 days in months with 31 days.

Actual Defective Behavior (Competition Mode):

Month end hardcoded to day 30, truncating the 31st day.

UI Recognition Clues:

End-of-month transactions on the 31st do not show up in monthly analytics.

Root Cause Analysis:

endDate = new Date(year, month - 1, 30, 23, 59, 59, 999) used instead of month end.

Exact Remediation / Solution:

Use new Date(year, month, 0, 23, 59, 59, 999) for calendar month end.

Corrected Behavior:

All transactions from day 1 to day 31 are included in reports.

Automated Regression Test:

server/src/tests/competition_bugs.test.js -> BUG-ADV-01

Post-Fix Verification:

Check October report includes expenses recorded on October 31.

BUG-ADV-02: Budget Utilization Subtracts Category Income From Limit

Difficulty: **Advanced** | Severity: High | Affected: Budget Calculations (BudgetController.getBudgets)

Reproduction Steps (from UI):

1. Set Groceries budget limit = ₹10,000.
2. Record expense = ₹4,000 in Groceries.
3. Record income/refund = ₹2,000 tagged under category "Groceries".
4. Observe budget: limit drops to ₹8,000 and utilization percentage inflates to 50%.

Expected Behavior:

Budget limits are fixed ceilings; income should not reduce the defined limit.

Actual Defective Behavior (Competition Mode):

Subtracts category income from monthlyLimit, shrinking the budget ceiling.

UI Recognition Clues:

Receiving refunds or income in a category lowers your configured monthly limit.

Root Cause Analysis:

limit = Math.max(1, budget.monthlyLimit - catIncome) in budgetController.js.

Exact Remediation / Solution:

Keep limit = budget.monthlyLimit; and handle income separately.

Corrected Behavior:

Budget limit remains exactly as configured by user.

Automated Regression Test:

server/src/controllers/budgetController.js

Post-Fix Verification:

Add income to budgeted category and verify limit is unchanged.

BUG-ADV-03: Category Line-Item Floor Rounding Discrepancy vs Grand Total

Difficulty: **Advanced** | Severity: Medium | Affected: Monthly Reports (ReportController.getMonthlyReport)

Reproduction Steps (from UI):

1. Create multiple transactions with decimal paise (e.g. ¹4,850.50, ¹1,850.75, ¹890.25).
2. Open Reports page.
3. Manually sum the Category Breakdown table rows and compare to Total Expenses card.
4. Notice a ¹1 to ¹3 discrepancy between table sum and KPI card.

Expected Behavior:

Sum of category items must exactly match the Grand Total card.

Actual Defective Behavior (Competition Mode):

Category items are truncated with Math.floor() while Grand Total uses standard rounding.

UI Recognition Clues:

Adding up the category breakdown table amounts does not equal the Total Expenses card.

Root Cause Analysis:

amount = Math.floor(c.total) in reportController.js aggregation format.

Exact Remediation / Solution:

Use standard decimal rounding: amount = Math.round(c.total * 100) / 100;

Corrected Behavior:

Table items and Grand Total card agree to the exact paisa.

Automated Regression Test:

server/src/controllers/reportController.js

Post-Fix Verification:

Sum category table amounts and verify match with Total Expenses card.

BUG-ADV-04: Filtered Pagination Calculates Total Pages Using Unfiltered Count

Difficulty: **Advanced** | Severity: Medium | Affected: Transactions Pagination (TransactionController.getTransactions)

Reproduction Steps (from UI):

1. Have 25 transactions across all categories.
2. Filter by category "Rent" (which only has 2 transactions).
3. Notice pagination shows "Page 1 of 3" instead of "Page 1 of 1". Clicking Next displays an empty page.

Expected Behavior:

Pagination totalPages must be derived from count of filtered records.

Actual Defective Behavior (Competition Mode):

Derives totalPages from total unfiltered records countDocuments({ userId }).

UI Recognition Clues:

Filtered transaction lists show multiple pages with blank pages on navigation.

Root Cause Analysis:

TransactionController uses countDocuments({ userId }) when filters are active.

Exact Remediation / Solution:

Always calculate totalCount = await Transaction.countDocuments(filter);

Corrected Behavior:

Pagination accurately reflects count of filtered results.

Automated Regression Test:

server/src/controllers/transactionController.js

Post-Fix Verification:

Filter transactions and verify total pages matches filtered record count.

BUG-ADV-05: Tablet Viewport (768px) Transaction Table Horizontal Overflow

Difficulty: **Advanced** | Severity: Medium | Affected: Responsive Layout (responsive.css)

Reproduction Steps (from UI):

1. Open Transactions page on a tablet or resize browser window to ~768px width.
2. Inspect the transactions table.
3. The table actions column forces fixed width with overflow-x: visible, spilling 24px past the card border.

Expected Behavior:

Table container should enable smooth horizontal scrolling without layout blowout.

Actual Defective Behavior (Competition Mode):

overflow-x: visible breaks the container card width on 768px screens.

UI Recognition Clues:

Transaction table spills over card edges on tablet screens, breaking alignment.

Root Cause Analysis:

CSS class .table-container.competition-overflow forces overflow-x: visible !important.

Exact Remediation / Solution:

Remove overflow-x override and ensure container maintains overflow-x: auto.

Corrected Behavior:

Table scrolls cleanly within container card on tablet viewports.

Automated Regression Test:

client/src/styles/responsive.css

Post-Fix Verification:

Resize viewport to 768px and verify table remains contained within card boundaries.